Intro

It's time to show students what they'll be creating this week!

Click **here** to play Ledge Throw!

How to Play

•Use arrow keys to move left / right
•Use the up key to jump
•Use the space key to throw a ledge in the direction you're facing

Objectives

•Reach the door at the end of the level

Current Code

What code is currently in our game?

Player

Setup

This code sets the gravity (how fast we fall down), makes sure our Player can't leave the world, and has code to allow touch users to play

Right

If we try to move right, this code will get called. It sets the facingRight variable to true, sets the speed of the Player to 200 and makes sure the Player looks right.

Left

The same concept as the Right function. This code will set the facingRight variable to false and play the left animation and move to the left at a speed of 200.

Ledge

Setup

We need to setup a few things for our Ledge actor. This code sets the value of x to 0 and y to it's starting **y** position. Since we don't want our Ledge to fall through the ground, it's not affected by gravity. No rotation ensures it doesn't ever turn / flip itself unexpectedly. Finally, making it a sensor ensures that it can't collide with any tiles or our Player.

Door

Next level

Self explanatory! This function takes us to the next level (when our Player hits it).

Lesson Plan

Ledge Throw

Oh no!! We're trapped! We need to rescue ourselves by throwing and jumping on ledges to reach the end of the level. We'll be `counting` actors, creating actors, and learning how to limit jumps with a `jumpCount` variable!

1. Define **movement** for our player

Click on menu: Ledger Throw

Click on Player, click on code tab, add new function

Hit save and test it out! Make sure your left and right movement works!

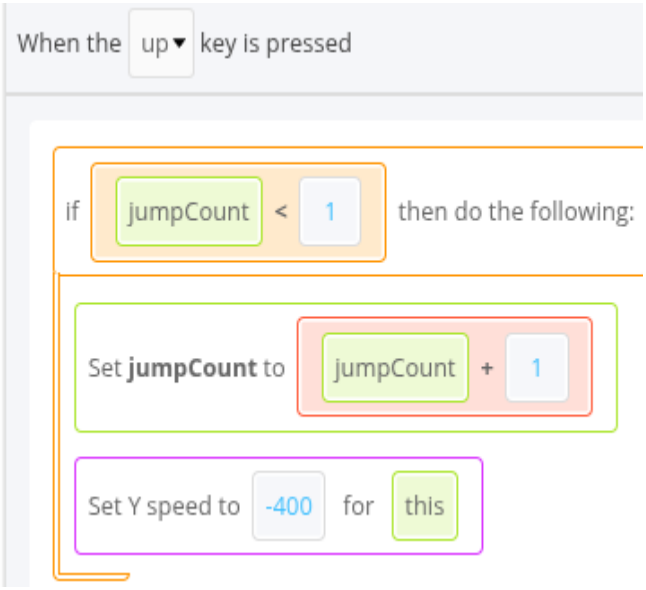
2. Define the JUMP!!

- New function `jump` on the Player — same "When this actor is updated" pattern as Step 1: check if the jump key (up) is pressed, and if so, call the jump behavior.
- Even though jumping is really just changing the y speed, this function needs more: we want to **limit** how many times the Player can jump in a row.
- Introduce a `jumpCount` variable.
- **The catch:** without a limit, players could jump straight to the top of the level — that's the "cheat" you're specifically preventing.
- Test: at this stage they'll notice they can only jump **once, ever**, in the whole game — that's expected and sets up Step 3.

Click on menu: Ledger Throw

Click on Player, click on code tab, add a new function `jump`.

Add a `jumpCount` variable

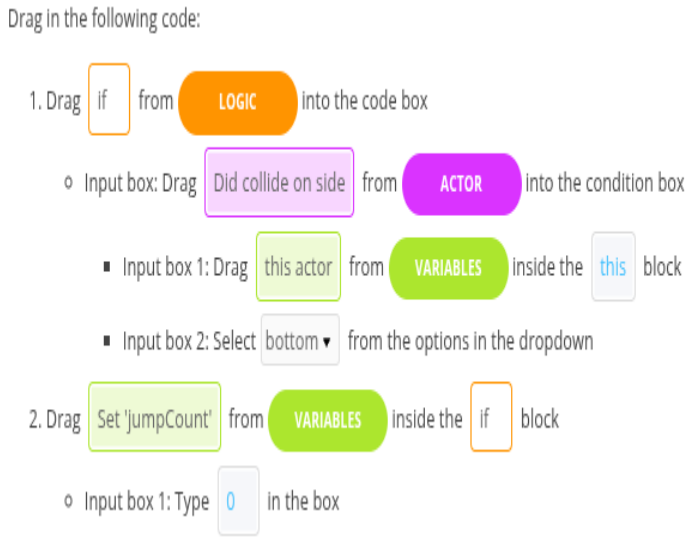


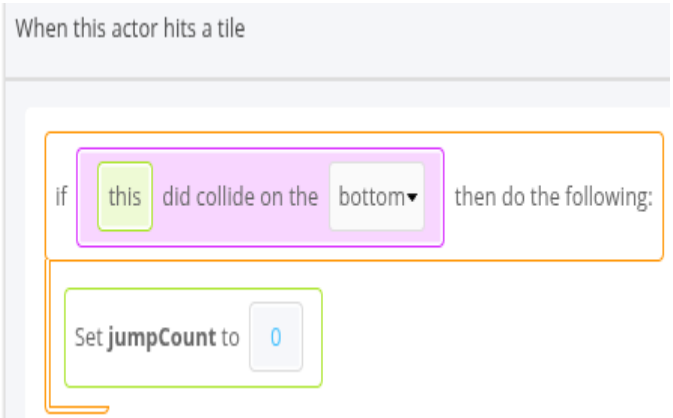
Hit save and try jumping around! Wait. We can only jump once in the entire game?

3. Define Reset function inside the Player actor

Click on menu: Ledger Throw

Click on Player, click on code tab, add a new function `Tile Reset`

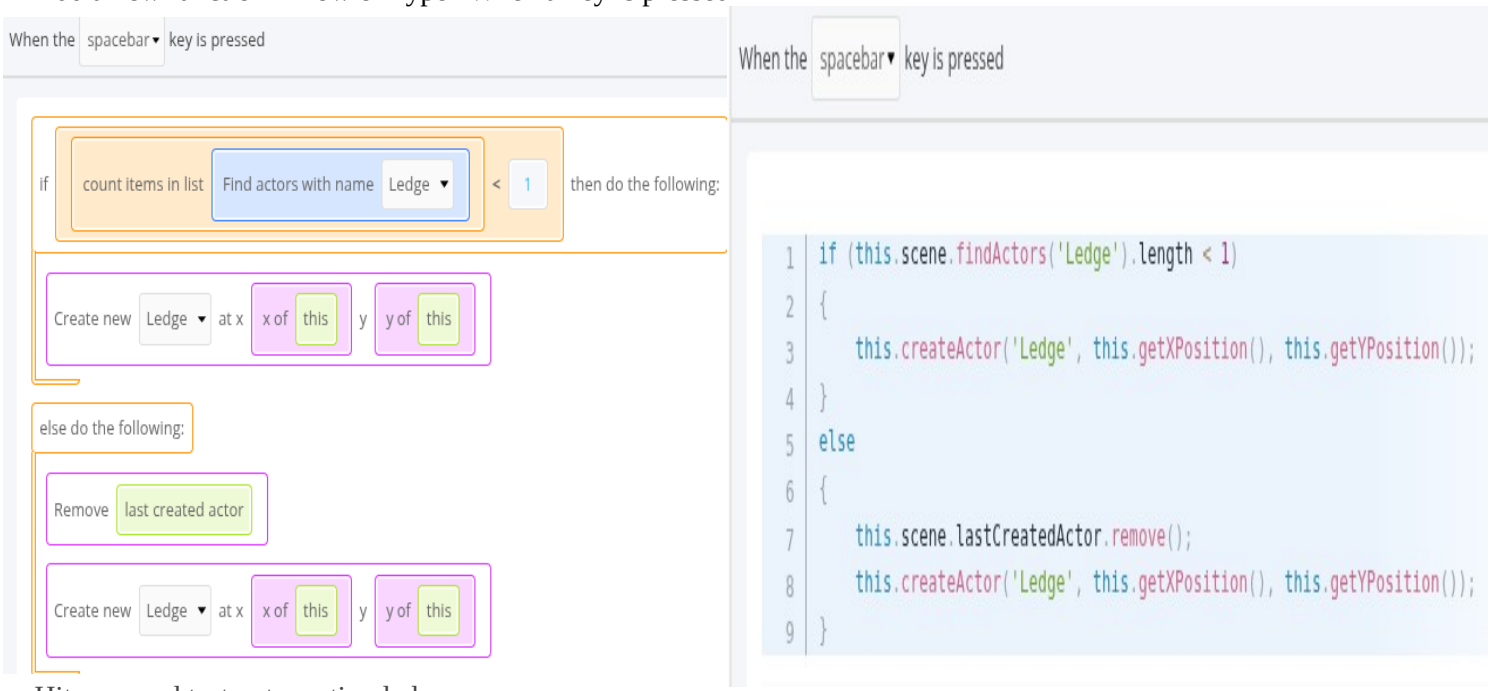




Test: Make sure you can jump up and down

4. Ledge Throwing

Add a new function Throw of Type “When a key is pressed”



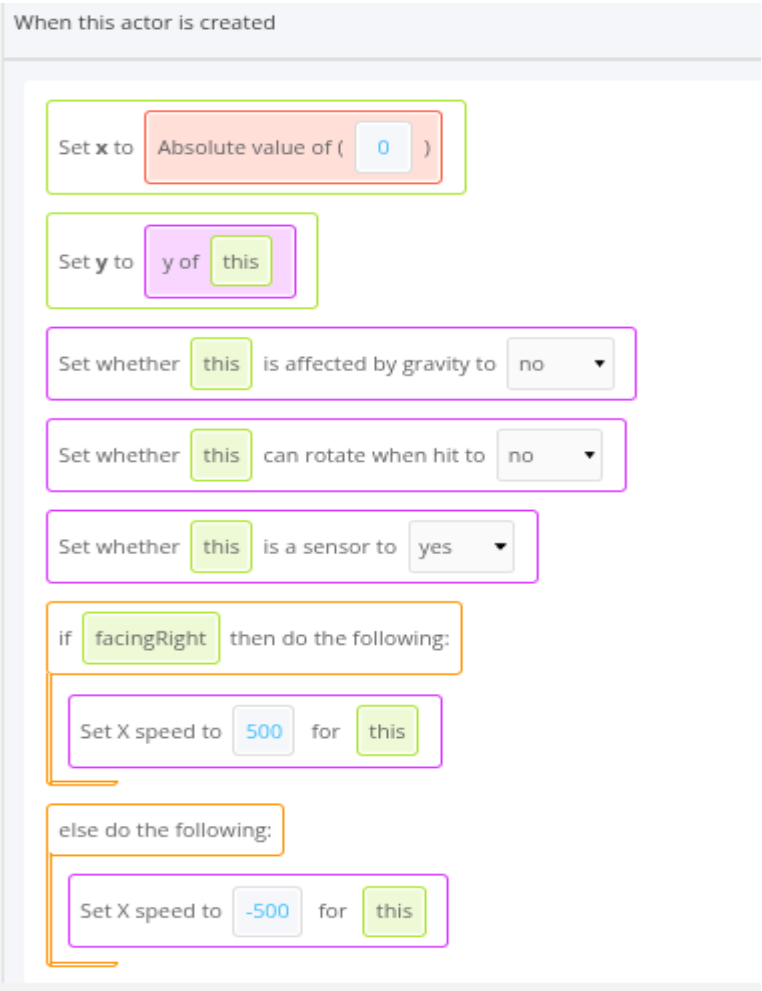
Hit save and test out creating ledges

5. The Right (and Left) Way [Ledge actor]

Now that we can create ledges, we need to make sure they fly in the right direction. To do that, we just need to use the facingRight variable. If we're facing right, we'll make it move with a speed of 500. If not, then it'll move left with a speed of 500.

We can do all that in our Ledge's **Setup** function (inside the Ledge actor)

1. Drag if from LOGIC into the code box
   - Input box: Drag Get 'facingRight' from VARIABLES into the condition box
2. Drag Set X speed from ACTOR inside the if block
   - Input box 1: Type 500 in the speed box
   - Input box 2: Drag this actor from VARIABLES inside the this block
3. Drag else from LOGIC into the code box
4. Drag Set X speed from ACTOR inside the else block
   - Input box 1: Type -500 in the speed box
   - Input box 2: Drag this actor from VARIABLES inside the this block



Save and test it out

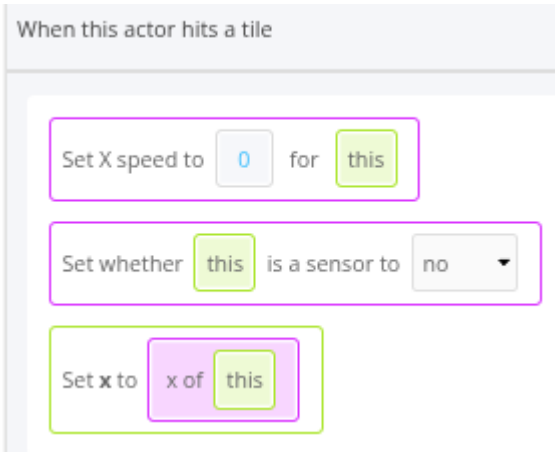
Ledge Too Fast or Slow?

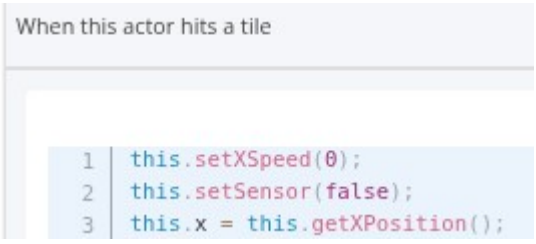
Try 1000 / −1000 for double speed, or 250 / −250 for half speed. Don't go too slow — it kills the pacing.

Step 6 Create a Stop function inside the Ledge actor

the ledge starts life as a **sensor** (so it doesn't collide with anything while flying). On stop, flip that off so it becomes a real, standable platform.

"a sensor can see through walls, a solid object can't."





Save and test it out

Step 7 Player Actor: Create a Ledge Reset function when this actor hits another actor

Jumping on the Ledge

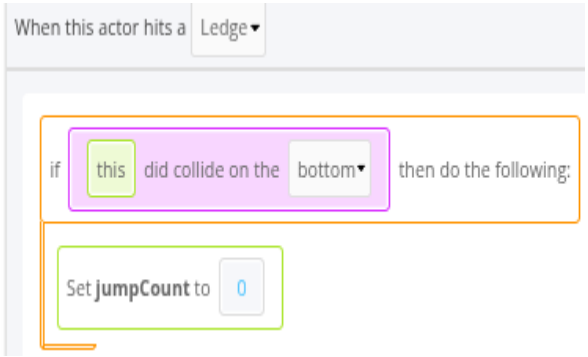
Since we only reset our jump when we hit a tile, we can't jump on the platform. Oh no! Let's make it so that our jumps also reset whenever our actor collides with a ledge with their feet.

- New `Ledge Reset` function: fires when the Player collides with a ledge.
- **Key detail — check that the collision is on the *bottom*** of the Player.
- *why does it matter which side we hit the ledge from?*
    - If you don't check "bottom only," touching the ledge with your **head** would also reset your jump — making the game trivially easy.
    - Checking "bottom" means the Player is actually *standing* on it, not just bumping it.

This is the single best "why" moment in the whole lesson — it's the same logic real games use to stop players exploiting collision boxes.

Drag in the following code:

1. Input box: Select Ledge ▾ from the options in the dropdown
2. Drag if from LOGIC into the code box
   - Input box: Drag Did collide on side from ACTOR into the condition box
     - Input box 1: Drag this actor from VARIABLES inside the this block
     - Input box 2: Select bottom ▾ from the options in the dropdown
3. Drag Set 'jumpCount' from VARIABLES inside the if block
   - Input box 1: Type 0 in the box



Why check if we're colliding on the bottom?

This means that the feet of our Player are touching the ledge.

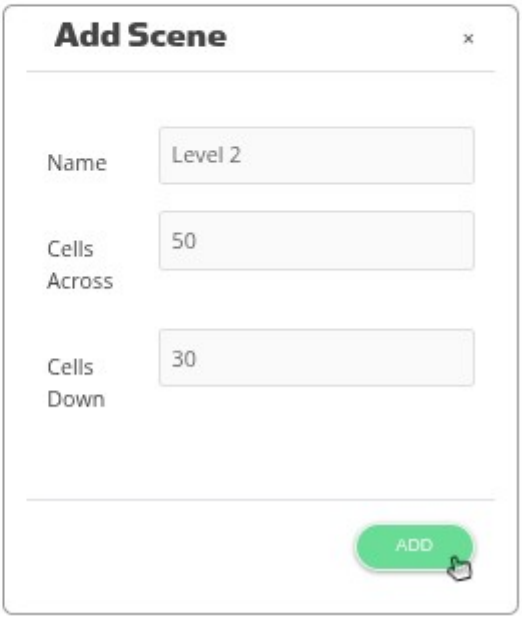
If we **didn't**, then as long as any part of the Player hits a ledge, we can jump again.

That means all we'd need to do is touch it with our head and our jumps would reset! That'd make the game TOO easy!So, when we check if we collided on the bottom, we're checking that we're standing on the platform and not that we're hitting it some other way where jumping wouldn't make sense.

Step 8. Another Escape

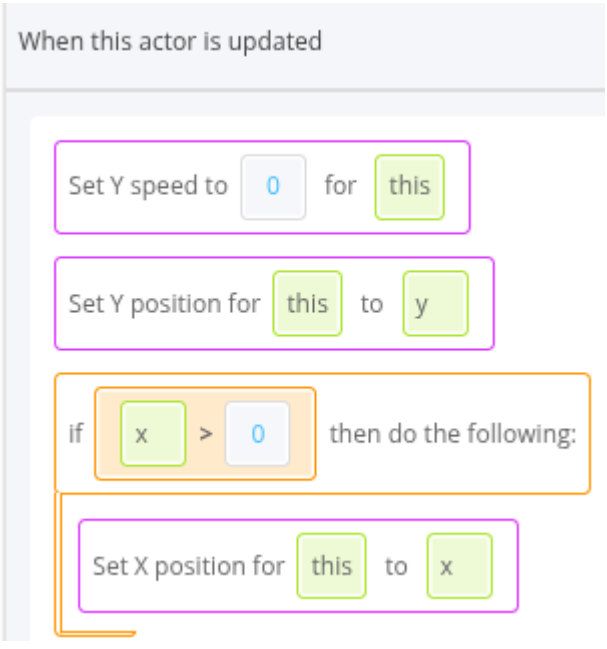
Click on the **Add Scene** button

Create a scene with the following details:



Step 9 Bug fix. Go to Stay Still function under Actor Ledge

- In the Ledge actor's `Stay Still` function: check if `x > 0`.
- Inside that `if`, set the ledge's x position back to the stored `x` value.
- **Why this bug happens:** the ledge actually *can* still be nudged left/right after landing (physics quirk), so this pins it to its landing spot permanently.

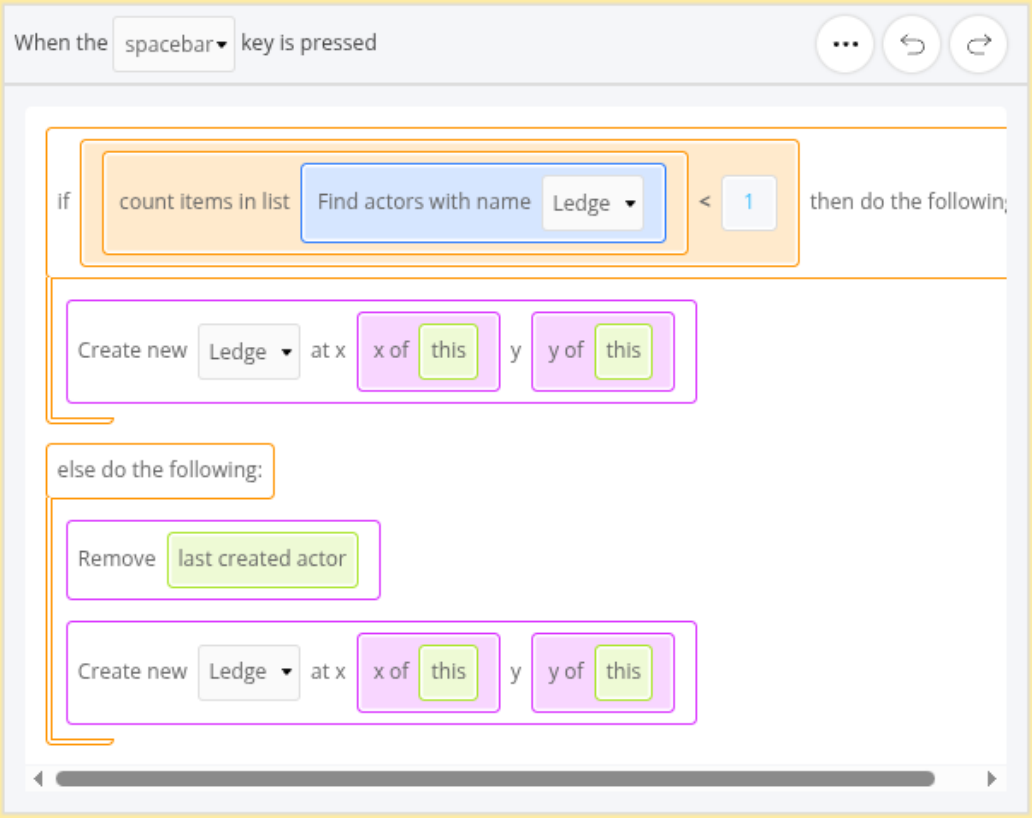


When this actor is updated

```
this.setYSpeed(0);
this.setYPosition(this.y);
if (this.x > 0)
{
    this.setXPosition(this.x);
}
```

Step 9 Wrap up

- **Question**: Looking at our Throw function, why do we have an else? Why can't we get rid of it?



Answer: We need to make sure that we **remove** the actor when we want to create a new one. We can't always remove the last created actor as that will either be the player or the door when you first start the level! So the if condition will only apply for the first time we throw a ledge.

Question: In our tile and ledge jump resets, why did we have to check if we collided on the bottom?

Answer: because an actor can collide with something from any side — top, bottom, left, right. Without the bottom-only check, a player could hug a wall and jump straight up, or reset their jump just by grazing a ledge with their head. Checking "bottom" confirms they're actually *standing* on something, which is the only case where resetting the jump makes sense.

Does anybody have any questions?